

Lecture 4: Median Finding Algorithm

Data Structures and Algorithms

August 29, 2012

1 Review of Quicksort

We previously looked at algorithms for sorting, mainly Quick Sort. The way Quick Sort works is:

1. Pick a pivot element in the array.
2. Compare each element in the array to the pivot.
3. Divide the array into two parts:
 - Elements less than the pivot.
 - Elements greater than the pivot.
4. Recurse on both parts to sort them.
5. Finally, put them together to get the larger sorted array.

The time taken by Quick Sort crucially depends on the position of the pivot element in the array. If the middle element (which we call the **median**) is chosen as the pivot, the algorithm gives the best performance. If the median is chosen for the pivot always, then the Quick Sort algorithm runs in $O(n \log n)$ time.

2 Objective and Definitions

Our objective is to find the median given an array of size n , and it should be fast—specifically in **linear time** $O(n)$.

Definitions

Rank of an element: The rank of an element is its position in the sorted array. Given an array, if you sort the array and find the position of the element in the sorted array, that is its rank.

- We assume that all elements in the array are distinct (just for simplicity).
- By the way rank is defined, the median of an array is an element of rank $\frac{n}{2}$.

The Goal

- **Input:** Array A
- **Output:** Median

Clearly, this can be done in $O(n \log n)$ time by sorting the array and picking the middle element. But we need a faster algorithm. How can we apply divide and conquer? How do we divide the array?

3 The Concept of an Approximate Median

Suppose you have somehow found the **approximate median**.

- **Approximate median:** An element x of rank greater than $\frac{n}{4}$ and less than $\frac{3n}{4}$.

$$\frac{n}{4} \leq \text{rank}(x) \leq \frac{3n}{4}$$

They are not quite the middle element, but they roughly fall around the middle. If someone gives you the approximate median, does this help find the exact median? The answer is yes.

Suppose x is the approximate median in array A . We can split the array into elements less than x (A_L) and elements greater than x (A_R).

- If $\text{rank}(x) > \frac{n}{2}$, then the exact median falls on the left side (A_L).
- If $\text{rank}(x) < \frac{n}{2}$, then the exact median falls on the right side (A_R).

The good thing about this partition is that both partitions have size *at least* $\frac{n}{4}$ (since there are at least $\frac{n}{4}$ elements less than x , and at least $\frac{n}{4}$ elements greater than x). This means we get rid of at least $\frac{1}{4}$ of the elements on each recursive step. The bigger part is of size at most $\frac{3n}{4}$. We have successfully shrunk the size of the array by a constant factor.

4 Generalizing the Problem

If we can find the approximate median in linear time $O(n)$, how much time does it take to find the exact median? The recurrence would look like:

$$T(n) = n + T\left(\frac{3n}{4}\right) \approx O(n)$$

However, there is a problem. We want to find the median of the big array A . If the median falls in A_R , what is its rank in A_R ? Let $|A_L|$ be the size of the left sub-array. The exact median has rank $\frac{n}{2}$ in the entire array A . When we look inside A_R , we have already bypassed $|A_L| + 1$ elements (the left array plus the pivot x). Thus, the rank we need to find in A_R is:

$$\text{rank in } A_R = \frac{n}{2} - |A_L| - 1$$

When we recurse, we are **not** trying to find the middle element of A_R . We are trying to find an element of a very specific, different rank. Along the way, we encounter a problem: we actually want to find an element of *some other rank*, not just rank $\frac{n}{2}$. Therefore, this is a **more general problem**. Finding the median is a special case of this generalized problem. Sometimes it is easier to solve a general problem than a specific problem.

The Generalized Problem Definition

- **Input:** Array A , target rank r .
- **Output:** An element of rank r in A .

By doing this, there is no problem with the recursion. We find an approximate median x , partition into A_L and A_R .

- If $r \leq |A_L|$, we recurse on A_L to find the element of rank r .
- If $r > |A_L| + 1$, we recurse on A_R to find the element of rank $r - |A_L| - 1$.

5 Finding the Approximate Median

How do we find the approximate median? This seems as difficult as finding the exact median. The solution is very clever.

1. Take the array A , split the elements into groups of 5. So we have $\frac{n}{5}$ groups, and each of them has 5 elements.
2. Find the **exact median** in each group. There are $\frac{n}{5}$ such medians.
3. Find the exact median of these $\frac{n}{5}$ medians. This element will serve as our approximate median.

Why is this an approximate median?

We started off finding the exact median, ended up finding the approximate median, and now we are finding the exact median again?

- First, we find exact medians of groups of 5. This takes $O(1)$ time per group, hence linear time overall.
- Second, we find the exact median of the $\frac{n}{5}$ medians. We can do this recursively! The size is reduced by a factor of 5, so we can recurse on them.

To convince you that this is an approximate median, let's arrange these elements. Sort each group of 5 in descending order top-to-bottom. Then, arrange the groups left-to-right such that their medians are sorted.

a_{15}	a_{25}	...	x	...	...	a_{k5}
a_{14}	a_{24}	...	...	...	...	a_{k4}
a_{13}	a_{23}	...	Median (x)	...	$a_{(k-1)3}$	a_{k3}
a_{12}	a_{22}	...	...	...	...	a_{k2}
a_{11}	a_{21}	...	...	...	...	a_{k1}

Notice that the elements in the top-right quadrant are all greater than x . Because the medians to the right are greater than x , and the two elements above those medians are even greater. This ensures that a guaranteed fraction (at least $\approx \frac{1}{4}$) of elements are strictly greater than x , and similarly, at least a quarter are strictly smaller.

6 The Algorithm: FindElement(r, A)

This function finds an element of rank r in array A .

1. Partition A into $\frac{|A|}{5}$ groups of size 5.
2. Find the median of each group of 5 and put them in an array B . The size of B is $\frac{|A|}{5}$.
3. **Find Approximate Median:**
 $x \leftarrow \text{FindElement}\left(\frac{|B|}{2}, B\right)$
4. **Partition A around x :**
 Elements $< x$ go to A_L . Elements $> x$ go to A_R .
5. **Recursive Cases:**
 - If $|A_L| = r - 1$, then return x .
 - If $|A_L| > r - 1$, return $\text{FindElement}(r, A_L)$.
 - If $|A_L| < r - 1$, return $\text{FindElement}(r - |A_L| - 1, A_R)$.

7 Algorithm Analysis

Now, how do we analyze the algorithm? Our expectation is that this procedure runs in linear time. Let's write down the recurrence relation and see.

Let $T(n)$ be the time taken on an array of size n .

- To find the median of each group, we can do a bubble sort which takes at most 25 comparisons per group. For $\frac{n}{5}$ groups, this is some constant time C_1n .
- Finding the approximate median (median of B) takes $T\left(\frac{n}{5}\right)$ time.
- Partitioning the array takes n time.
- The final recursive step on either A_L or A_R takes at most $T\left(\frac{3n}{4}\right)$ because size of $|A_L| \geq \frac{n}{4}$ (and equivalently $|A_R| \geq \frac{n}{4}$), which ensures that the sub-array we recurse on is at most $\frac{3n}{4}$.

Therefore, the recurrence relation is:

$$T(n) \leq C_1n + T\left(\frac{n}{5}\right) + n + T\left(\frac{3n}{4}\right)$$

$$T(n) \leq (C_1 + 1)n + T\left(\frac{n}{5}\right) + T\left(\frac{3n}{4}\right)$$

Because $\frac{1}{5} + \frac{3}{4} = \frac{4}{20} + \frac{15}{20} = \frac{19}{20} < 1$, the sum of the fractions inside the recurrences is less than 1. This guarantees that the geometric series bounded by the tree of recursive calls shrinks at every level, yielding a total running time of $O(n)$.